An Experimental Study of Task-Oriented Dialogue Systems

Ahmed Moustafa
Matriculation Number: 3968514
Supervisor: Dr. Nurul Fithria Lubis
Heinrich Heine University Düsseldorf

Abstract

Task-oriented spoken dialogue systems help users complete goals, such as finding a restaurant or arranging a hotel stay, through natural-language interaction. A complete system must represent the user's words, identify intents and slot values, maintain the dialogue state, choose the next action, and generate a suitable response. This report studies these decisions through six connected experiments, from word embeddings and a modular ConvLab3 pipeline to learned language understanding, dialogue state tracking, reinforcement learning, and LLM-based agents. The emphasis is on the implementation choices, observed results, and limitations of each approach.

1 Word Embeddings

One-hot vectors treat every word as an independent symbol and do not express similarity between related words. Dense word embeddings instead learn compact vectors from context. This lets a dialogue model share evidence between related slot values, such as different locations or price ranges. In Exercise 1, I studied Word2Vec (Mikolov et al., 2013), FastText (Bojanowski et al., 2017), and a small custom Skip-Gram model. I first trained Word2Vec from scratch on the general-domain *text8* corpus. The domain-specific models used the provided set of 1,364 restaurant utterances from the second Dialogue State Tracking Challenge (DSTC2) (Henderson et al., 2014).

The *text8* model produced 100-dimensional vectors for 71,290 word types. Its nearest neighbours for *apple* were mainly technology and company terms, as shown in Table 1. In a candidate comparison, *fruit* was selected when no technology term was present, but *computer* was selected after it was added. This illustrates a limitation of static embeddings: different senses of a word share one vector, so the most common corpus context can dominate the representation.

Model	Query	Nearest neighbours
text8 W2V	*apple*	macintosh (.792), intel (.716), ibm (.715)
DSTC2 W2V	*west*	south (.897), east (.864), north (.831)
DSTC2 W2V	*moderate*	expensive (.544), cheap (.521)
FastText	*inexpensive*	expensive (.991), give (.590), moderate (.547)
Custom Skip-Gram	*cheap*	moderate, expensive, price

Table 1: Selected nearest neighbours from the saved Exercise 1 run. W2V abbreviates Word2Vec. Parentheses show cosine similarity; the custom model did not save similarity scores.

On DSTC2, the Gensim Word2Vec model used the default CBOW architecture with 300 dimensions, a context window of 3, a minimum count of 1, and 1,000 epochs. It grouped location words and price values, although the similarity between opposite price values shows that related context does not imply equal meaning. FastText could also construct a vector for the unseen word *inexpensive* from character subwords. However, it ranked the antonym *expensive* first with a similarity of .991. Subword information therefore improves coverage, but spelling overlap can be risky when a dialogue system interprets user preferences.

Finally, the custom 100-dimensional, full-softmax Skip-Gram model used a window of 5 and was trained for 100 epochs. Its saved mean batch loss fell from 4.233 at epoch 0 to 3.361 at epoch 90, with most of the improvement occurring by epoch 10. Its nearest neighbours formed useful groups for price, location, and cuisine. These results are qualitative: the notebook contains one saved run without fixed random seeds or a held-out evaluation. I also excluded its sentence-similarity values because that helper averaged characters rather than tokenized words.

2 Toolkit and Corpora

Task-oriented systems predict structured actions that help a user reach a specific goal. In a modular pipeline, natural language understanding (NLU) maps an utterance to dialogue acts, dialogue state tracking (DST) updates the current slot values, the policy selects a system act using this state and the database result, and natural language generation (NLG) realizes the act as text. ConvLab3 standardizes the interfaces between these modules and uses `PipelineAgent` to execute them in sequence (Zhu et al., 2023). This design makes individual components easy to replace, although an error in an early module can propagate through the complete pipeline.

2.1 Simple MultiWOZ 2.1 and the Unified Format

Simple MultiWOZ 2.1 is the hotel-and-restaurant subset of MultiWOZ 2.1 (Eric et al., 2020). The local unified archive contains 2,503 dialogues: 2,162 for training, 164 for validation, and 177 for testing. General-domain acts represent functions such as greeting, thanking, and ending a conversation.

The unified format consists of dialogue records, an ontology, and a database interface. The ontology defines domains, slots, possible categorical values, intents, dialogue acts, and the state structure. Each dialogue contains a user goal and turns labelled with the speaker, utterance, and dialogue acts; the format can also attach user states, database results, and booked entities. Dialogue acts may be categorical intent–slot–value labels, non-categorical values linked to spans in the utterance, or binary acts without values. These annotations support all four pipeline modules: utterances supervise NLU acts, dialogue histories supervise states, states and database results supervise policy actions, and system acts are paired with responses for NLG.

2.2 Baseline ConvLab3 Pipeline

I completed the supplied `interact.py` baseline. The system agent used a `BERTNLU` checkpoint trained on user turns, rule-based state tracking, the supplied maximum-likelihood policy, and template-based NLG. I disabled manual entity-name features to match the saved policy's state-vector dimensions. `PipelineAgent` connected the modules, while the command-line loop initialized each session, limited interaction to 40 turns, and stopped when the user entered `bye`.

In the recorded interaction in Appendix B, the system moved from a restaurant request to a hotel request and returned relevant entities. However, it recommended an expensive restaurant after the user requested a cheap one, produced awkward template phrases, and did not confirm either booking request. This shows that a plausible response is not sufficient for reliable goal completion. Improvements should validate policy actions against the tracked constraints and database results, represent booking status explicitly, request clarification when information is uncertain, and refine the response templates. More policy training on constraint changes and multi-domain booking dialogues would also target the observed failures.

3 Natural Language Understanding with BERTNLU

3.1 Task and Model

In Exercise 3, I completed and trained ConvLab3's BERTNLU model (Devlin et al., 2019; Zhu et al., 2023). Preprocessing produced 11,806 training, 1,007 validation, and 1,064 test user turns, with 70 intent labels and 17 BIO slot tags. Each sample supplies WordPiece identifiers and attention masks for the current utterance and up to three earlier utterances. Categorical and binary dialogue acts become a multi-label intent target, while non-categorical values are represented by token-level BIO tags. The model outputs 70 intent logits per utterance and 17 slot logits per token. Training minimizes the sum of weighted binary cross-entropy for intents and cross-entropy for slot tags.

3.2 Training Configuration

The configuration controls the encoder and tokenizer, context use, fine-tuning, hidden size, dropout, batch size, learning rate, sequence limit, optimizer, seed, device, and training length. I used the supplied `prajjwal1/bert-mini` encoder with the `bert-base-uncased` tokenizer because it was small enough for local training while retaining contextual representations. I fine-tuned the encoder and context branch for 5,000 updates on a laptop GPU, using batch size 64, learning rate 10^{-4}, dropout 0.1, a 40-token utterance limit, three context turns, and seed 2019. Validation was run every 500 updates, and the checkpoint with the highest validation F1 was retained.

3.3 Evaluation and Analysis

Table 2 reports corpus-level dialogue-act scores from the completed evaluator. Exact match is stricter: every predicted act for an utterance must equal its reference set. Test F1 was 0.43 points above validation F1, and exact match rose by 1.23 points, so there was no observed generalization drop between these splits.

Split	P	R	F1	Exact
Validation	84.79	89.46	87.06	73.49
Test	85.76	89.30	87.49	74.72

Table 2: BERTNLU dialogue-act results in percent. P and R denote precision and recall; Exact is utterance-level exact-match accuracy.

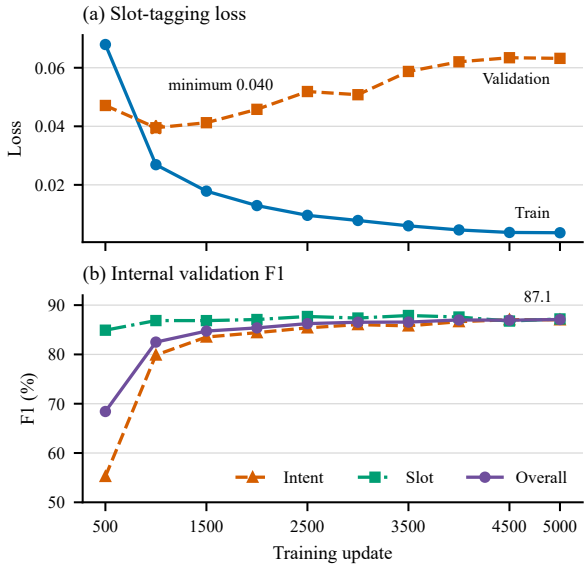

Figure 1: BERTNLU learning dynamics at 500-step checkpoints. The curves are unsmoothed; training loss averages the preceding 500 batches, while validation loss covers the complete split.

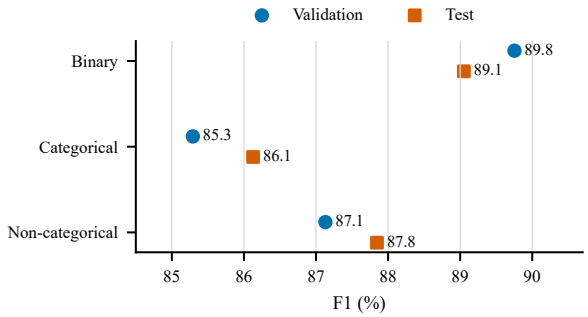

Figure 2: Standardized validation and test F1 by dialogue-act type.

Recall exceeded precision on both splits. Figure 2 shows that categorical acts were consistently weakest. Their test F1 was 86.13, compared with 87.85 for non-categorical spans and 89.05 for binary acts. Errors included missing an area expressed indirectly through context, dropping the multiword value *Eastern European*, and confusing two- and three-star categories.

Figure 1 shows a different pattern for loss and F1. Training slot loss fell from 0.0679 at step 500 to 0.0036 at step 5,000, whereas validation slot loss reached its minimum at step 1,000 and then rose from 0.0395 to 0.0632. Internal validation F1 nevertheless increased from 68.42 to 87.09, so F1-based checkpoint selection retained the final model. The loss divergence suggests mild overfitting. Other limitations are the single seeded run, fixed three-turn context, a fixed intent threshold, and dependence on exact span annotations; preprocessing skipped one malformed restaurant span. Useful next steps are multi-seed evaluation, threshold tuning, early stopping, a constrained BIO decoder, and correction of noisy spans.

4 Dialogue State Tracking with TripPy

4.1 Architecture, Inputs, and Objective

4.2 Training Dynamics

4.3 Ablation Study and Analysis

5 Dialogue Policy with PPO

5.1 PPO Setup and Policy Variants

5.2 RL Training Results

5.3 Interaction Quality

6 Agentic Task-Oriented Dialogue

6.1 Free-Form Web-Enabled LLM

6.2 Structured Modular Prompting

6.3 Comparison and Deployment Risks

7 Conclusion

References

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Mihail Eric, Rahul Goel, Shachi Paul, Abhishek Sethi, Sanchit Agarwal, Shuyang Gao, Adarsh Kumar, Anuj Goyal, Peter Ku, and Dilek Hakkani-Tur. 2020. MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 422–428, Marseille, France. European Language Resources Association.

Michael Heck, Carel van Niekerk, Nurul Lubis, Christian Geishauser, Hsien-Chin Lin, Marco Moresi, and Milica Gašić. 2020. TripPy: A triple copy strategy for value independent neural dialog state tracking. In *Proceedings of the 21st Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 35–44, 1st virtual meeting. Association for Computational Linguistics.

Matthew Henderson, Blaise Thomson, and Jason D. Williams. 2014. The second dialog state tracking challenge. In *Proceedings of the 15th Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGDIAL)*, pages 263–272, Philadelphia, PA, U.S.A. Association for Computational Linguistics.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2016. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Qi Zhu, Christian Geishauser, Hsien-Chin Lin, Carel van Niekerk, Baolin Peng, Zheng Zhang, Shutong Feng, Michael Heck, Nurul Lubis, Dazhen Wan, Xiaochen Zhu, Jianfeng Gao, Milica Gašić, and Minlie Huang. 2023. ConvLab-3: A flexible dialogue system toolkit based on a unified data format. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 106–123, Singapore. Association for Computational Linguistics.

A Implementation and Reproducibility

The exercise implementations are collected in the *spoken-dialogue-systems* GitHub repository. The separate HHU GitLab course forks are ConvLab3 and TripPy. The completed Exercise 2 pipeline is available in the ConvLab3 interaction script. Exercise 3 is represented by the training script, evaluator, and run configuration. It was executed on Windows in the Conda base environment with Python 3.13.5, PyTorch 2.8.0, Transformers 4.49.0, and an NVIDIA GeForce RTX 3050 Laptop GPU, using seed 2019. The Exercise 3 figures can be regenerated from the checked-in metric extracts with the plotting script.

B Example Dialogues

B.1 Baseline ConvLab3 Pipeline

The user goal was to find a cheap restaurant in the south, find a hotel in the same area with free parking, and book both places. The transcript is reproduced from the saved Exercise 2 interaction, with speaker labels and punctuation normalized for readability.

System: Hello, how can I help you?

User: I am looking for a restaurant in the south part of town.

System: It is located in the south. There are 9 available restaurants. That is a chinese restaurant. They are in the expensive price range. What type of food would you like?

User: I want cheap food.

System: Would you like to try the good luck chinese food takeaway? It is located in the south. I have 9 options for you. It is chinese food. They are in the expensive price range. I would suggest the good luck chinese food takeaway.

User: Can you also find me a hotel in the south?

System: There are 1 of those.

User: Does it have free parking?

System: How about the lensfield hotel? They are located in the south. We have 1 such places. It is a hotel.

User: Book a table at the chinese food takeaway restaurant at 20:00 today and one room at lensfield hotel today at 22:00.

System: Does the the lensfield hotel work? It is located at 53–57 lensfield road. It is indeed in the south. The wifi is included. This is an expensive hotel. It is a hotel. Is there anything else I can help you with today?

User: Yes, book a room.

System: You are welcome. Is there anything else I can help you with today?

User: Bye.

System: Goodbye!

B.2 PPO Dialogue Policies

B.3 Free-Form LLM and Boundary Probes

B.4 Structured-Prompt LLM

C Final Structured Prompt